MediCare - Online Doctor Appointment System

A comprehensive Django-based web application for managing doctor appointments, built with modern web technologies and best practices.

🎓 Project Information

This project was developed as part of the **10th Quera Software Engineering Bootcamp** with Django, showcasing a complete full-stack web application with modern development practices and team collaboration.

👥 Development Team

Mentor:

- Ali Julaee Rad

Team Members:

- **Amirhossein Khorram niaki** - Product Owner
- **Amin Hamifar** - Scrum Master
- **Navid Hosseini** - Tech Lead
- **Morteza Khanbabaie** - Developer
- **Khosro Parichehre** - Developer

This project demonstrates effective team collaboration using Agile methodologies, with clear role definitions and responsibilities throughout the development process.

🎯 Bootcamp Learning Outcomes

This project showcases the comprehensive skills learned during the Quera Django Bootcamp:

- **Django Framework Mastery**: Advanced Django concepts including models, views, templates, and admin
- **Class-Based Views**: Modern Django architecture with CBVs for maintainable code
- **Database Design**: Complex relationships, migrations, and data modeling
- **Authentication & Authorization**: Multi-role user system with secure access control
- **API Development**: RESTful endpoints and AJAX integration
- **Frontend Integration**: Tailwind CSS for modern, responsive UI design
- **Testing**: Comprehensive unit testing with Django TestCase
- **Project Management**: Agile methodologies, Git workflow, and team collaboration
- **DevOps Basics**: Environment configuration, deployment preparation, and database seeding

🚀 Features

Core Functionality

- **User Management**: Multi-role system (Admin, Doctor, Patient) with authentication

- **Doctor Profiles**: Specialized doctor profiles with specialties, fees, and ratings
- **Appointment Booking**: Easy appointment scheduling with real-time availability
- **Wallet System**: Integrated payment system for patients
- **Availability Management**: Automated timeslot generation based on doctor availability windows
- **Review System**: Patient reviews and ratings for doctors

Technical Features

- **Modern UI**: Built with Tailwind CSS for responsive, beautiful interfaces
- **Class-Based Views**: Clean, maintainable Django architecture
- **Database Seeding**: Comprehensive test data generation
- **Email Integration**: Appointment confirmation emails
- **Admin Interface**: Full Django admin integration
- **Security**: CSRF protection, user authentication, and role-based access

🏗️ Project Structure

```
AP/
├── core/                    # Django project settings
├── users/                   # User authentication and profiles
├── doctor/                  # Doctor management and availability
├── booking/                 # Appointment booking system
├── wallet/                  # Payment and wallet management
├── theme/                   # Tailwind CSS styling
├── templates/               # Global templates
├── static/                  # Static files (CSS, JS, images)
├── scripts/                 # Database seeding scripts
└── requirements.txt         # Python dependencies
```

🛠️ Installation & Setup

Prerequisites

- Python 3.8+
- Node.js (for Tailwind CSS)
- Virtual environment

1. Clone and Setup

```
git clone <repository-url>
cd AP
python -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate
pip install -r requirements.txt
```

2. Environment Configuration

Create a `.env` file in the project root:

```
SECRET_KEY=your-secret-key-here
DEBUG=True
```

3. Database Setup

```
python manage.py migrate
python scripts/seed.py  # Populate with sample data
```

4. Tailwind CSS Setup

```
cd theme
npm install
npm run build
cd ..
```

5. Run Development Server

```
python manage.py runserver
```

Visit `http://127.0.0.1:8000` to see the application.

👥 User Roles & Access

Admin

- Full system access
- User management
- Doctor profile management
- System configuration

Doctor

- Manage availability windows
- View appointments
- Update profile information
- Generate timeslots automatically

Patient

- Browse doctors

- Book appointments
- Manage wallet
- View appointment history
- Leave reviews

🔧 Key Models

DoctorAvailability

- Defines when doctors are available (day of week, time windows)
- Configurable visit duration
- Automatic timeslot generation

Timeslot

- Individual appointment slots
- Auto-generated from availability windows
- Booking status tracking

Appointment

- Patient-doctor appointments
- Status management (scheduled, confirmed, completed, cancelled)
- Integration with wallet system

Wallet

- Patient payment system
- Transaction history
- Balance management

🧮 Usage Guide

For Patients

1. **Register/Login**: Create account with patient role
2. **Browse Doctors**: Search by specialty, name, or availability
3. **Book Appointment**: Select available timeslot and confirm
4. **Manage Wallet**: Add funds and pay for appointments
5. **View History**: Track past and upcoming appointments

For Doctors

1. **Create Profile**: Set specialty, fees, and basic information
2. **Set Availability**: Define when you're available (e.g., Mon-Fri 9-5)
3. **Manage Timeslots**: System auto-generates slots based on availability
4. **View Appointments**: See scheduled patients

For Admins

1. **User Management**: Create and manage user accounts
2. **Doctor Management**: Approve and manage doctor profiles
3. **System Monitoring**: View appointments, transactions, and system health

UI/UX Features

- **Responsive Design**: Works on desktop, tablet, and mobile
- **Modern Interface**: Clean, professional medical theme
- **Intuitive Navigation**: Easy-to-use navigation with role-based menus
- **Real-time Updates**: Dynamic content updates without page refresh
- **Accessibility**: WCAG-compliant design patterns

Security Features

- **Authentication**: Secure user login with email/phone verification
- **Authorization**: Role-based access control
- **CSRF Protection**: Cross-site request forgery prevention
- **Input Validation**: Comprehensive form validation
- **SQL Injection Protection**: Django ORM prevents SQL injection

Database Seeding

The project includes a comprehensive seeding script that creates:

- Sample users (admin, doctors, patients)
- Doctor profiles with specialties and fees
- Availability windows for doctors
- Generated timeslots for the next 30 days
- Sample appointments and reviews
- Wallet balances for patients

Run with: `python scripts/seed.py`

Deployment

Production Settings

1. Update `core/settings/prod.py` with production values
2. Set `DEBUG=False`
3. Configure database (PostgreSQL recommended)
4. Set up static file serving
5. Configure email settings

Environment Variables

```
SECRET_KEY=your-production-secret-key
DEBUG=False
DATABASE_URL=postgresql://user:password@host:port/dbname
```

```
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_HOST_USER=your-email@gmail.com
EMAIL_HOST_PASSWORD=your-app-password
```

🧪 Testing

Comprehensive Test Suite

The project includes a robust testing framework with **79 comprehensive unit tests** covering:

- **Model Tests**: All core models (User, Doctor, Appointment, Wallet, etc.)
- **View Tests**: Class-based views, authentication, and authorization
- **Admin Tests**: Admin dashboard functionality and user management
- **Integration Tests**: End-to-end workflow testing
- **Edge Cases**: Error handling and boundary conditions

Running Tests

```
# Run all tests
python run_tests.py

# Run specific test modules
python manage.py test core.tests
python manage.py test doctor.tests
python manage.py test booking.tests
python manage.py test wallet.tests
python manage.py test users.tests
```

Default Login Credentials

- **Admin**: username: `admin`, password: `password123`
- **Doctor**: username: `dr_smith`, password: `password123`
- **Patient**: username: `patient1`, password: `password123`

Test Data

The seeding script creates realistic test data including:

- 3 doctors with different specialties
- 3 patients with wallet balances
- 1,164 generated timeslots
- Sample appointments and reviews

🏆 Technical Achievements

This project demonstrates advanced Django development skills:

- **79 Unit Tests**: Comprehensive test coverage ensuring code reliability
- **Class-Based Architecture**: Modern Django patterns with CBVs throughout
- **Complex Database Design**: Multi-app relationships with proper foreign keys and constraints
- **Role-Based Security**: Sophisticated authentication and authorization system
- **Real-time Features**: AJAX integration for dynamic user experience
- **Admin Customization**: Extensive Django admin customization for content management
- **Responsive Design**: Mobile-first approach with Tailwind CSS
- **Database Seeding**: Automated test data generation for development and testing
- **Code Quality**: Clean, maintainable code following Django best practices

📈 Future Enhancements

- **Real-time Notifications**: WebSocket integration for live updates
- **Calendar Integration**: Google Calendar/Outlook sync
- **Video Consultations**: Telemedicine integration
- **Mobile App**: React Native or Flutter mobile application
- **Analytics Dashboard**: Advanced reporting and analytics
- **Multi-language Support**: Internationalization
- **Payment Gateway**: Stripe/PayPal integration

🤝 Contributing

1. Fork the repository
2. Create a feature branch
3. Make your changes
4. Add tests if applicable
5. Submit a pull request

🆘 Support

For support and questions:

- Create an issue in the repository
- Contact the development team
- Check the documentation

🎓 Quera Bootcamp Project

This project was developed as part of the **10th Quera Software Engineering Bootcamp** with Django, demonstrating the team's mastery of modern web development practices and collaborative software engineering.

Built with ♥ using Django, Tailwind CSS, and modern web technologies.